

Deadlock — Design Document

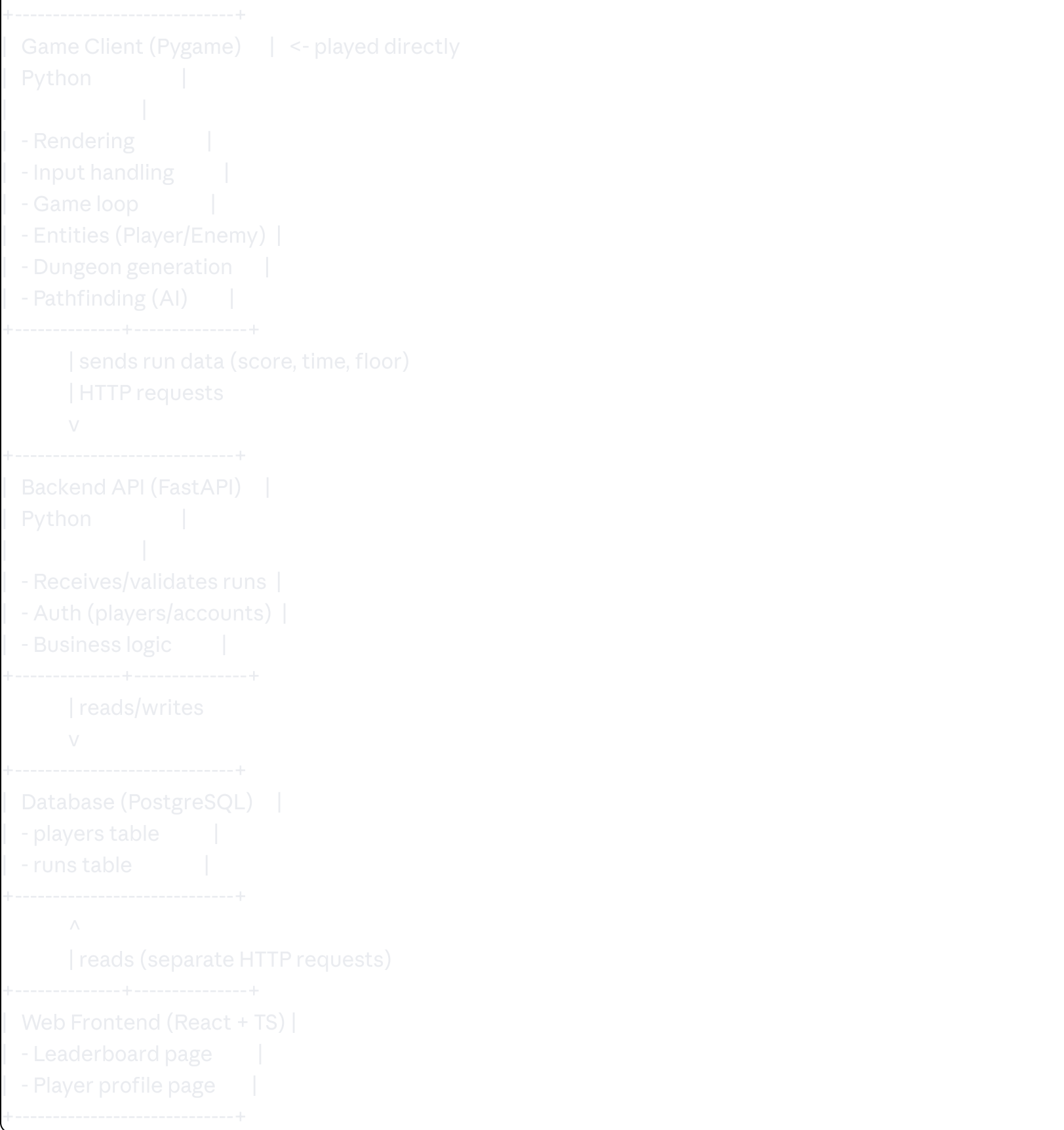
What it is

A single-player roguelike dungeon crawler, built in layers. Each layer teaches a distinct, resume-relevant CS concept. The player fights through procedurally generated dungeon floors against AI-controlled enemies, with runs saved to a leaderboard.

Core principle: every phase ends with something playable. No pile of half-built systems — at the end of each phase, the game runs and shows the new thing working.

Architecture





Three independent programs talking over HTTP — client, API, database, and a web view. This is the same basic shape as most real production software.

Build phases

Phase	What gets built	New concept learned
1. Core loop	Player moves on a grid, walls, one enemy that chases and fights you	Game loop, OOP (Entity/Player/Enemy classes), grid-based collision
2. Procedural generation	Replace the hand-built map with an algorithm that generates a new dungeon layout every run	Algorithm design (BSP or cellular automata)
3. Enemy AI / pathfinding	Enemies navigate around walls intelligently instead of walking into them	Graph algorithms (A*), state machines
4. Persistence	Save/load runs locally, maybe SQLite	Basic data persistence, file/DB I/O
5. Backend API	FastAPI service that accepts run submissions and stores them in PostgreSQL	REST API design, schema design, validation
6. Web frontend	React + TypeScript site showing the live leaderboard, reading from the API	Frontend/backend separation, async data fetching, typed languages
7. Deployment	Docker containers, deployed to a free-tier host (Railway/Render/Fly.io)	DevOps, environment config, shipping something real

What makes this a strong portfolio piece

Not that it's a game — each phase forces a real engineering decision that's defensible in an interview:

Why BSP over cellular automata for dungeon generation? (Phase 2)

Why A* over BFS/Dijkstra for enemy pathfinding? (Phase 3)

How was the run/player schema designed to avoid duplicate or cheated submissions? (Phase 5)

How does the frontend update the leaderboard without a full page reload? (Phase 6)

Those are software engineering questions, not "I made a game" trivia.

Working model

This project is being built by the student, not generated. Claude's role is to explain concepts, review code, and ask questions that check understanding — not to write the implementation. Each phase should end with the student able to explain *why* each choice was made, not just that it works.

Status log

- ☐ Phase 1 — Core loop
- ☐ Phase 2 — Procedural generation
- ☐ Phase 3 — Enemy AI / pathfinding
- ☐ Phase 4 — Persistence
- ☐ Phase 5 — Backend API
- ☐ Phase 6 — Web frontend

